

Shubham Kathavale

INTERVIEW PREPARATION & RESUME CHEAT SHEET

1. The 90-Second Elevator Pitch (Self-Introduction)

When an interviewer opens with "Tell me about yourself," use this structured template to establish your core technical and personal traits:

- **Hook (Present):** "I am a Computer Science & Engineering student at MIT-WPU, Pune, specializing in system automation, multi-agent AI environments, and full-stack development. My main focus is bridging the gap between intelligence engines and robust developer tools."
- **Background (Past):** "I completed my Diploma in Computer Engineering with First Class Distinction from B.L. Patil Polytechnic, MSBTE. This built my core computing and database foundations. Following my diploma, I secured direct second-year entry into MIT-WPU where I've continued to build systems from the ground up."
- **Key Projects (Proof):** "Most notably, I built J.A.R.V.I.S.—a proactive Personal AI Operating System using a PySide6 Qt GUI, a glassmorphic Next.js dashboard, and a local agentic brain with vector RAG search (using FAISS) and SQLite cognitive memory graphs. I've also built Evokn Fitness, a full-stack macro tracker, and developed custom multiplayer scripts on MySQL backend for GTA V servers."
- **Behavioral Edge (Discipline):** "What truly defines my execution style is consistency and self-discipline. I lost 30 kg during a dedicated fitness phase—a challenge that built high standards of planning, metrics tracking, and commitment that I bring directly into my engineering and code quality."
- **The Goal (Conclusion):** "I am looking for an internship or junior developer role where I can solve database scale issues, coordinate system pipelines, and contribute to production-ready products."

2. Project Deep-Dives (STAR Method)

A. J.A.R.V.I.S. — Personal AI Operating System

Situation: Needed to evolve a standard reactive voice assistant into a proactive desktop operating companion that understands user files, goals, and habits.

Task: Create an event-driven system coordinating specialized agents (Task, Knowledge, Memory, Vision, Desktop) concurrently, managing RAG and local permanent state.

Action: Built a PySide6 loop hosting Next.js QWebView at 60 FPS. Configured RAG using local FAISS vector databases on repository codebase directories. Created permanent cognitive context using SQLite.

Key Challenge: Headless coordination of agents without infinite loop callbacks.

Solution: Implemented a central orchestrator pattern routing events asynchronously via local loop callbacks.

Result: Under 5% CPU idle load and under 300ms local RAG retrieval latency.

B. Evokn Fitness — Full-Stack Fitness Tracker

Situation: Needed an intuitive application to log workout analytics, calories, and weight targets.

Task: Develop a responsive full-stack platform with dashboard charts, macro logs, and secure user profiles.

Action: Programmed frontend in React.js/Tailwind. Handled storage and profiles via Firebase Firestore and Authentication.

Key Challenge: Firestore state synchronization conflicts on rapid consecutive macronutrient inputs.

Solution: Implemented client-side optimistic updates and debounced database write operations.

Result: Eliminated state thrashing; achieved sub-second client latency and smooth mobile responsiveness.

C. Noisecity RP — GTA V FiveM Server Scripting

Situation: Managed custom roleplay gameplay features and economy transactions for an active multiplayer community.

Task: Store and link player assets and vitals dynamically without introducing game thread latency.

Action: Wrote game logic using Lua and FiveM APIs. Integrated server endpoints with a remote MySQL database.

Key Challenge: High-frequency transactional writes from 100+ concurrent players causing locks.

Solution: Cached player vitals in server memory and implemented structured bulk commits every 5 minutes.

Result: Reduced database connection overhead by 80%, maintaining server tick rates at a smooth 60 FPS.

3. Core Academic Concepts (Syllabus Prep)

MIT-WPU exams prefer structured answers: **Definition, Example, Working/Key Points, Applications, Conclusion, Memory Trick**. Use these structures during technical questions:

A. Design & Analysis of Algorithms (DAA)

- **Dynamic Programming (DP):** Breaks problems into overlapping subproblems, solves each once, and stores results (Memoization/Tabulation) to avoid duplicate computation. (e.g. Fibonacci, 0/1 Knapsack, Floyd-Warshall).
Memory Trick: **DP = Divide and Preserve** (Store sub-answers).
- **Greedy Algorithms:** Makes the locally optimal choice at each step to find a global optimum. Fast but doesn't guarantee a global optimum. (e.g. Dijkstra, Fractional Knapsack, Prim's).
Memory Trick: **Greedy = Grab Immediately** (Short-sighted but fast).

B. Database Management Systems (DBMS)

- **ACID Properties:**
 - *Atomicity:* Transactions are all-or-nothing.
 - *Consistency:* Keeps database schema valid from start to end.
 - *Isolation:* Concurrent transactions do not overwrite or conflict.
 - *Durability:* Once committed, data is safe from crashes.*Memory Trick:* **ACID = Atomic, Consistent, Isolated, Durable**.
- **Normalization Levels:**
 - *1NF:* Atomic column values (no nested array arrays/repeating groups).
 - *2NF:* 1NF + no partial dependency (every attribute depends on the entire candidate key).
 - *3NF:* 2NF + no transitive dependency (non-keys rely only on keys, not other non-keys).

C. Data Structures & Microprocessors

- **Hash Tables:** Achieves average $O(1)$ search and insertion. Collision resolution is handled via Chaining (linked lists) or Open Addressing (probing).
- **Microprocessor registers (MMA):** Small, high-speed storage locations within the CPU. Work with instruction pipelines, memory fetch phases, and hardware interrupts.

4. Behavioral Questions & Personal Discipline

- **How do you manage deadlines and focus?**
"My 30 kg weight loss journey required strict self-tracking, consistent execution, and objective goal-setting. I translate this exact strategy into software engineering by breaking goals down into sprints, using issue trackers,

and logging systems metrics."

- **How do you learn new technologies?**

"I completed my Diploma with distinction while simultaneously self-studying Web/AI systems. For JARVIS OS, I learned PySide6 Qt bindings, asynchronous event queues, and vector index RAG architectures. I learn by researching documentation, compiling prototypes, and debugging implementation limits."